Manipulación de Cadenas y Punteros en C/C++

Control de accesos operando directamente sobre la memoria RAM

Alexis Maier

E.E.S.T. N° 1 "Eduardo Ader" — Vicente López

Laboratorio de Programación (LPR) — 5° Año 3° División — Grupos A-B

Profesor: Prof. Mansilla Muñoz York Elías

Fecha: 01 de octubre de 2026

Introducción

La actividad 4 introdujo el trabajo directo sobre la memoria mediante punteros, el mecanismo por el cual C y C++ se apartan de la administración automática de datos. La consigna pidió un programa de control de accesos que capturara la edad del usuario directamente en la variable a través de un puntero, evaluara la mayoría de edad sin copiar el dato y, en la segunda parte, manipulara una cadena de caracteres enlazada por puntero, mostrando su contenido, su dirección física, su primer carácter y su longitud.

Desarrollo

Declaración del puntero

Se declara la variable *int edad = 0;* y a continuación *int* p = &edad;*. El operador **&**, llamado dirección de, devuelve la dirección física de la variable y esa dirección queda almacenada en el puntero *p*. La variable *p* no contiene el valor de la edad, sino la ubicación en la RAM donde ese valor reside.

Captura mediante desreferencia

La lectura del dato se realiza con *cin >> *p;*. El asterisco es el operador de desreferencia: indica que el valor debe escribirse en la celda apuntada, es decir, directamente sobre *edad*. Es fundamental comprobar que el asterisco esté presente: escribir *cin >> p;* intentaría sobrescribir la dirección hexadecimal almacenada en el puntero, destruyendo el vínculo con la variable original.

Evaluación y presentación

El condicional trabaja sobre **p*, el valor alojado en memoria. Según el resultado, el sistema responde [ACCESO APROBADO] o [ACCESO RECHAZADO], imprime la edad con **p* y muestra la dirección física con *p*, que en consola se visualiza en notación hexadecimal, por ejemplo 0x61fe1c. La misma variable, mostrada de dos maneras distintas, demuestra la diferencia entre el valor alojado y su ubicación.

Manipulación de cadenas

Antes de leer el nombre se ejecuta *cin.ignore();* para eliminar el salto de línea pendiente en el búfer, que de lo contrario interrumpiría la lectura. Luego *char nombre[50];* reserva un arreglo de 50 caracteres en memoria y *cin.getline(nombre, 50);* captura la cadena completa respetando el límite. Al asignar *char* punteroCadena = nombre;*, el puntero adopta automáticamente la dirección del primer elemento, lo que permite imprimir la cadena entera, la dirección base con la conversión *(void*)*, el primer carácter con **punteroCadena* y la cantidad de caracteres con *strlen(nombre)*.

Cuestionario Técnico de Control

1. Diferencia entre & y *

El operador **&** actúa sobre una variable y devuelve su dirección de memoria; el operador ***** se aplica a un puntero y devuelve el valor almacenado en esa dirección. Con la analogía de la

consigna, *&* entrega la ubicación del casillero y * abre el casillero para leer o escribir su contenido.

2. Motivo de la conversión a void*

En C++, el operador << no reconoce un *char** como un valor imprimible por defecto, porque interpreta ese tipo como una cadena de texto. Aplicar la conversión *(void*)* convierte el puntero en un puntero genérico que el flujo de salida trata como dirección, mostrando así el valor hexadecimal.

3. Función de cin.ignore()

Cuando se mezclan *cin* >> y *cin.getline()*, el salto de línea que dejó el lector numérico permanece en el búfer de entrada. *cin.ignore()* descarta ese carácter residual, de modo que la lectura siguiente no se quede con una cadena vacía. Es un problema clásico de la entrada por consola y reaparece en todas las actividades con cadenas.

4. Arreglo frente a puntero

Un arreglo de caracteres reserva un bloque contiguo de memoria cuyos elementos se acceden con aritmética de punteros; el nombre del arreglo, al declararse, se convierte automáticamente en un puntero al primer elemento. Un puntero es una dirección almacenada en una variable y puede apuntar a cualquier ubicación, no solo a un arreglo propio.

5. Autoevaluación

El momento de mayor comprensión fue entender que *cin >> p* sin desreferencia no produce un error visible sino un daño silencioso: el programa sigue funcionando, pero la dirección guardada ya no corresponde a *edad*. Esa comprensión motivó a revisar cuidadosamente cada uso del asterisco en el código.

Conclusión

La actividad demandó comprender que la memoria es un recurso direccionable y no una caja abstracta. El uso de *&* y * permite escribir y leer directamente en una celda concreta, lo que resulta indispensable cuando se trabaja con hardware, sensores o estructuras de tamaño definido. Comprender que una cadena no es más que un arreglo de caracteres con un terminador nulo, y que un puntero es simplemente una dirección, cierra la brecha entre el modelo mental de alto nivel y el comportamiento real del microprocesador, sentando las bases para el modelado con estructuras del taller siguiente.

Referencias

cppreference.com. (s. f.). *Pointers*. Referencia del lenguaje C++. https://en.cppreference.com/w/cpp/language/pointer

cplusplus.com. (s. f.). *Pointers*. Tutorial de C++. https://cplusplus.com/doc/tutorial/pointers/

Microsoft Learn. (s. f.). *Punteros, direcciones y aritmética de punteros*. https://learn.microsoft.com/es-es/cpp/cpp/pointers

cppreference.com. (s. f.). *std::strlen*. Referencia de C++. https://en.cppreference.com/w/c/string/byte/strlen